Documentación técnica

Arquitectura, modelo de datos y API del sistema · EDT 5.1.2

1. Arquitectura del sistema (EDT 3.3.1)

Aplicación **web local** de tres capas, sin servidor externo:

```
Navegador (HTML + CSS + JavaScript)
        │  fetch() — API JSON
        ▼
Flask (Python) — app.py   ←  Lector USB = teclado (barra de escaneo)
        │  sqlite3
        ▼
Base de datos SQLite — stock.db
```

COMPONENTE	TECNOLOGÍA	ARCHIVO
Backend / API	Python · Flask	`app.py`
Persistencia	SQLite (stdlib `sqlite3`)	`db.py` · `stock.db`
Frontend	HTML + CSS + JavaScript vanilla	`templates/` · `static/`
Códigos de barras	python-barcode (EAN-13 / CODE128 en SVG)	`requirements.txt`

2. Modelo de datos (diseño de base de datos — EDT 2.2.3 / Matriz RACI)

productos

CAMPO	TIPO	DESCRIPCIÓN
id	INTEGER PK	Identificador
codigo_barras	TEXT UNIQUE	Código EAN-13 (o CODE128) escaneado
nombre	TEXT	Descripción del producto
categoria	TEXT	Almacén, Bebidas, Lácteos…
precio	REAL	Precio de venta
stock_actual	INTEGER	Unidades disponibles
stock_minimo	INTEGER	Umbral de reposición

CAMPO	TIPO	DESCRIPCIÓN
proveedor_id	INTEGER FK	Proveedor asociado

proveedores

`id`, `nombre` (único), `contacto`.

movimientos

`id`, `producto_id` (FK), `tipo` (entrada / salida), `cantidad`, `motivo`, `fecha`. Cada venta genera salidas; cada recepción o ingreso genera entradas.

lista_reposicion

`id`, `producto_id` (FK), `cantidad_sugerida`, `estado` (**anotado → pedido → recibido / reclamo**), `fecha`. Estados y transiciones según el cursograma “Control Stock”.

3. API (endpoints)

Productos

MÉTODO	RUTA	FUNCIÓN
GET	`/api/productos?q=`	Listar / buscar productos
GET	`/api/productos/buscar?codigo=`	Buscar por código escaneado
POST	`/api/productos`	Alta de producto
PUT	`/api/productos/<id>`	Edición
DELETE	`/api/productos/<id>`	Baja

Proveedores

MÉTODO	RUTA	FUNCIÓN
GET	`/api/proveedores`	Listar (con conteo de productos)
POST	`/api/proveedores`	Alta
DELETE	`/api/proveedores/<id>`	Baja (si no tiene productos)

Venta y ingresos

MÉTODO	RUTA	FUNCIÓN
POST	`/api/venta`	Valida stock, descuenta y registra salidas; anota en reposición si llega al mínimo
POST	`/api/ingreso`	Ingreso directo: suma stock y registra entradas

Lista de reposición

MÉTODO	RUTA	FUNCIÓN
GET	`/api/reposicion?estado=`	Listar por estado
POST	`/api/reposicion`	Anotar producto (por id o código)
POST	`/api/reposicion/anotar-minimos`	Anotar todos los que están en el mínimo
POST	`/api/reposicion/generar-pedido`	Anotados → pedido (por proveedor)
POST	`/api/reposicion/<id>/recepcion`	Recepción: ingreso (suma stock) o reclamo

Tablero, alertas e impresión

MÉTODO	RUTA	FUNCIÓN
GET	`/api/dashboard`	Indicadores y últimos movimientos
GET	`/api/alertas`	Faltantes, bajo mínimo y reclamos
GET	`/api/movimientos?limit=`	Historial de movimientos
GET	`/api/etiqueta/<id>`	Etiqueta SVG con código de barras

Vistas

`/` (Tablero) · `/venta` · `/ingreso` · `/reposicion` · `/productos` · `/proveedores` · `/etiquetas` · `/manual` · `/documentacion` · `/imprimir/reposicion`

4. Integración con el lector de códigos de barras

El lector USB funciona como **teclado (keyboard wedge)**: escribe el código en el campo `#scan-input` de la barra superior y envía **Enter**. El archivo `static/js/scanner.js` detecta la tecla

Enter, limpia el campo y dispara un evento `scan` en la página activa; cada pantalla decide qué hacer con el código (agregar a la venta, buscar el producto, anotar en reposición…).

5. Instalación y ejecución

```
pip install -r requirements.txt   # Flask + python-barcode
python app.py                     # crea stock.db con datos de ejemplo
# abrir http://127.0.0.1:5000
```

Para reiniciar los datos de ejemplo: borrar `stock.db` y ejecutar `python db.py`.